

Настройка физики автомобиля – советы и примеры

Понимание модели из `8_drive_physics.md` и TUNING

Вы используете **модель m1.5**, где движение автомобиля рассчитывается в мировых координатах (`(x, y)` – позиция, `(vx, vy)` – скорость).

Параметры `road_s` и `road_d` используются только как ориентиры для системы дорог, поэтому к ним не нужно привязывать управление.

Код `TUNING` задаёт дискретизацию и базовые характеристики. Важные моменты:

- `dt = 1/60` – фиксированный шаг симуляции. Большее значение уменьшит нагрузку, но снизит точность.
- **Максимальная скорость (`max_speed`) и макс. скорость заднего хода (`max_reverse_speed`)** определяют верхний предел скорости. Это не реальное ограничение – оно просто обрезает вектор скорости. В реальном физическом симуляторе ограничения скорости следуют из баланса тяги и сопротивления, как описывает классический учебник по физике автомобилей: сопротивление воздуха пропорционально квадрату скорости и вместе с силой сцепления формирует точку равновесия, при которой машина перестаёт ускоряться ¹. Поэтому сильная обрезка `max_speed` создаёт ощущение «стены» и может ухудшать управление.
- `accel`, `brake` и `coast_decel` – коэффициенты ускорения, торможения и замедления при отпуске газа. В реальной модели ускорение можно получать из баланса сил тяги, сопротивления и трения шин ². В вашем случае это простые числовые коэффициенты, поэтому изменение `brake` (он существенно больше `accel`) даёт более аркадный контроль.
- **Параметры поворота и сцепления:**
 - `steer_rate` – скорость изменения направления. Если машина ощущается «вялой», увеличьте это значение, но будьте готовы уменьшить его при высокой скорости, иначе игрок будет слишком резко поворачивать.
 - `grip` и `side_friction` – множители бокового трения; вместе со временем (`dt`) они определяют, насколько быстро гасятся боковые скольжения. Большие значения дают более «прилипчивую» машину, маленькие – больше заноса. В репозитории **ArcadeVehiclePhysics** отмечено, что для аркадного стиля величина бокового трения (stickiness) определяет, насколько шина «скользит»; нулевая stickiness даёт полный занос, а 1 – полное сцепление ³.
 - `side_slip_speed_mult` определяет, насколько сцепление падает при высокой скорости. Большие значения дают сильный занос на больших скоростях и заставляют водителя плавно входить в повороты; уменьшите его, если нужно больше стабильности.
 - `handbrake_grip_mult` уменьшает сцепление при ручнике (0.4 значит, что сцепление снижено до 40 %).
- **Off-road:** `offroad_grip_mult` и `offroad_slowdown` добавляют «драг» при съезде с дороги. Правильно подобранные значения позволяют игроку чувствовать разницу между асфальтом и песком.

Эти параметры образуют набор простых **«магических констант»**. Классический учебник по моделированию автомобилей подчеркивает, что такие коэффициенты допускается настраивать «по ощущениям» – игра не обязана имитировать реальность; главное, чтобы движение было приятным ⁴.

Почему «не управляется»: возможные причины

1. **Недостаток бокового трения.** Малые значения `grip` и `side_friction` приводят к сильным заносам. В реальных моделях боковую силу создаёт **скольжение колеса** – при повороте резина деформируется, возникает боковое усилие, которое сначала растёт, затем падает при больших углах скольжения ⁵. Ваш код упрощает эту зависимость до постоянного множителя. Попробуйте увеличить `grip` (3–4) и `side_friction` (8–10) и уменьшить `side_slip_speed_mult` (1–2).
2. **Слишком большой `steer_rate`.** Если машина реагирует рывками, уменьшите `steer_rate` или связывайте его со скоростью: на большой скорости ограничивайте поворот (популярная практика, описанная во многих руководствах по автомобильной физике).
3. **Резкая граница между дорогой и оффроадом.** Если игрок улетает в кювет, возможно, `offroad_grip_mult` слишком мал, а `offroad_slowdown` слишком велик. Увеличьте `offroad_grip_mult` до 4–5 и уменьшите `offroad_slowdown` до 1.0, чтобы съезд не приводил к немедленной остановке.
4. **Резкий переход при отпуске газа.** Ваша модель обрезает скорость «в ноль» (`coast_decel` = 25). В реальной модели сопротивление пропорционально скорости и возводится в квадрат ⁶. Чтобы сделать поведение более плавным, уменьшите `coast_decel` и добавьте небольшое квадратичное сопротивление (`drag`), чтобы скорость уменьшалась быстрее на высоких скоростях.

Рекомендации по настройке

1. **Определитесь с желаемым опытом.** В статье «Programming Vehicles in Games» отмечено, что главное – решить, какой опыт вы хотите передать: реалистичный симулятор или аркадный драйв ⁷. От этого зависит выбор параметров.
2. **Используйте итеративный подход.** Автор той же статьи описывает, как разработчик постепенно добавлял «магические числа», искал «скользкость» и постоянно тестировал игру ⁸. Вам придётся крутить параметры, играть и исправлять возникающие проблемы.
3. **Разделяйте продольные и боковые силы.** Классическая теория рекомендует отдельно рассчитывать продольные силы (тяга, тормоз, сопротивление воздуха и качению) и боковые силы (сцепление в повороте) ⁹. Таким образом можно проще контролировать поведение автомобиля: менять трение шин, не затрагивая ускорение.
4. **Добавьте сопротивление воздуха и роликовое сопротивление.** Для более «натурального» замедления используйте формулы:
5. **Сопротивление воздуха:** $F_{drag} = -C_{drag} * v * |v|$ ¹⁰.
6. **Сопротивление качению:** $F_{rr} = -C_{rr} * v$ ¹¹. Подбор `C_drag` и `C_rr` позволит машине естественно переставать ускоряться без искусственного ограничения `max_speed`.

Руководства и примеры

- **ArcadeVehiclePhysics (Unity).** Репозиторий описывает упрощённую аркадную физику. В разделе о боковом трении сказано, что **«боковое трение определяет, насколько шины**

будут скользить в сторону; эта реализация даёт хороший контроль, но оставляет небольшой занос» ¹² . Для настройки предлагается изменять коэффициент *tyre stickiness* (0 – нет сцепления; 1 – полное сцепление) ¹³ . Аналогичный подход можно реализовать, варьируя `grip` .

- **Car Physics for Games — «Magic constants»**. Эта статья советует использовать фиксированные коэффициенты сопротивления (`C_drag` и `C_rr`) и настраивать их «по ощущениям», особенно в аркадных играх ⁴ . Она объясняет, как баланс тяги и сопротивления определяет максимальную скорость, и что сопротивление воздуха растёт с квадратом скорости ¹⁰ .
- **Edy's Vehicle Physics (EVP)**. Пользовательский справочник к коммерчному пакету для Unity показывает, что свойства «**Tire Friction**» и «**Aero Downforce**» прямо влияют на ускорение и сцепление: коэффициент сцепления шин >1 увеличивает ускорение, а пониженное — снижает ¹⁴ ; аэродинамическая прижимная сила, зависящая от квадрата скорости, увеличивает нагрузку на колёса и улучшает сцепление на высокой скорости ¹⁵ . Аналогичные механизмы можно добавить в вашу модель.

Открытые игры для изучения механики

Если вы хотите ознакомиться с готовой реализацией и протестировать управление, попробуйте игры с открытым исходным кодом:

Игра	Описание	Полезные особенности
TORCS – The Open Racing Car Simulator	Старейший симулятор, написанный на C++, развивается с 1997 года. Свободно распространяется под GPL. Поддерживает Windows, Linux и другие платформы ¹⁶ . На SourceForge отмечено, что TORCS предоставляет реалистическую физическую модель, множество автомобилей и трасс и модульную архитектуру ¹⁷ .	Реалистичная физика; удобна для изучения. Можно изменить параметры и наблюдать эффект.
SuperTuxKart	Аркадный карт-рейсер с открытым кодом (GPL), кроссплатформенный, активная разработка — стабильная версия 1.5 вышла в октябре 2025 ¹⁸ .	Аркадная физика с поддержкой дрифта и нитро; хорошо показывает, как добиться «приятного» управления и удерживать баланс между реализмом и весельем.
ArcadeVehiclePhysics / ArcadeCarPhysics	Открытые фреймворки для Unity. README описывает принцип бокового трения, стабилизацию при боковых ударах и разделение физической модели и визуальной анимации ³ .	Код прост для изучения, можно увидеть, как реализованы <code>steer_rate</code> , <code>lateral friction</code> и <code>drift</code> .

Другие проекты, такие как **Speed Dreams** (форк TORCS), **Trigger Rally** и **VDrift**, также предоставляют свободный доступ к исходникам и движку. Они могут послужить хорошей отправной точкой для вдохновения и тестирования.

Что делать дальше

1. **Поиграйте в открытые игры.** Запустите TORCS или SuperTuxKart и обратите внимание, как разработчики регулируют скорость, занос, торможение и поворот. Это поможет понять, какие ощущения нравятся вам.
2. **Итеративно регулируйте параметры.** Начните с увеличения `grip` и `side_friction`, уменьшения `side_slip_speed_mult` и откорректируйте `steer_rate`. После каждой правки запускайте игру и оценивайте управление.
3. **Добавьте сопротивление воздуха/качению и прижимную силу.** Это даст более плавное поведение и избавит от искусственного `max_speed`.
4. **Сохраняйте баланс.** Помните, что в аркадных играх допустимы «магические числа»: цель – приятное управление, а не строгая физика ⁷. Поэтому подгоняйте параметры под собственные ощущения, но опирайтесь на общие принципы: отдельно контролировать продольные и боковые силы, применять сопротивление, учитывать влияние скорости на занос.

В итоге, улучшение поведения автомобиля потребует времени и экспериментов. Используйте существующие руководства и открытые проекты как ориентир, но не бойтесь «крутить параметры» – в аркадных играх это нормальная практика.

1 2 4 6 9 10 11 Car Physics

<https://rsms.me/etc/car-physics/>

3 12 13 GitHub - benmcinnes/ArcadeVehiclePhysics: A framework for creating an arcade inspired physics system for vehicles in Unity

<https://github.com/benmcinnes/ArcadeVehiclePhysics>

5 7 8 Programming Vehicles in Games

https://wassimulator.com/blog/programming/programming_vehicles_in_games.html

14 15 Edy's Vehicle Physics – User Guide – Edy's Projects

<https://www.edy.es/dev/vehicle-physics/user-guide/>

16 TORCS - Wikipedia

<https://en.wikipedia.org/wiki/TORCS>

17 TORCS - The Open Racing Car Simulator download | SourceForge.net

<https://sourceforge.net/projects/torcs/>

18 SuperTuxKart - Wikipedia

<https://en.wikipedia.org/wiki/SuperTuxKart>